Assignment 6

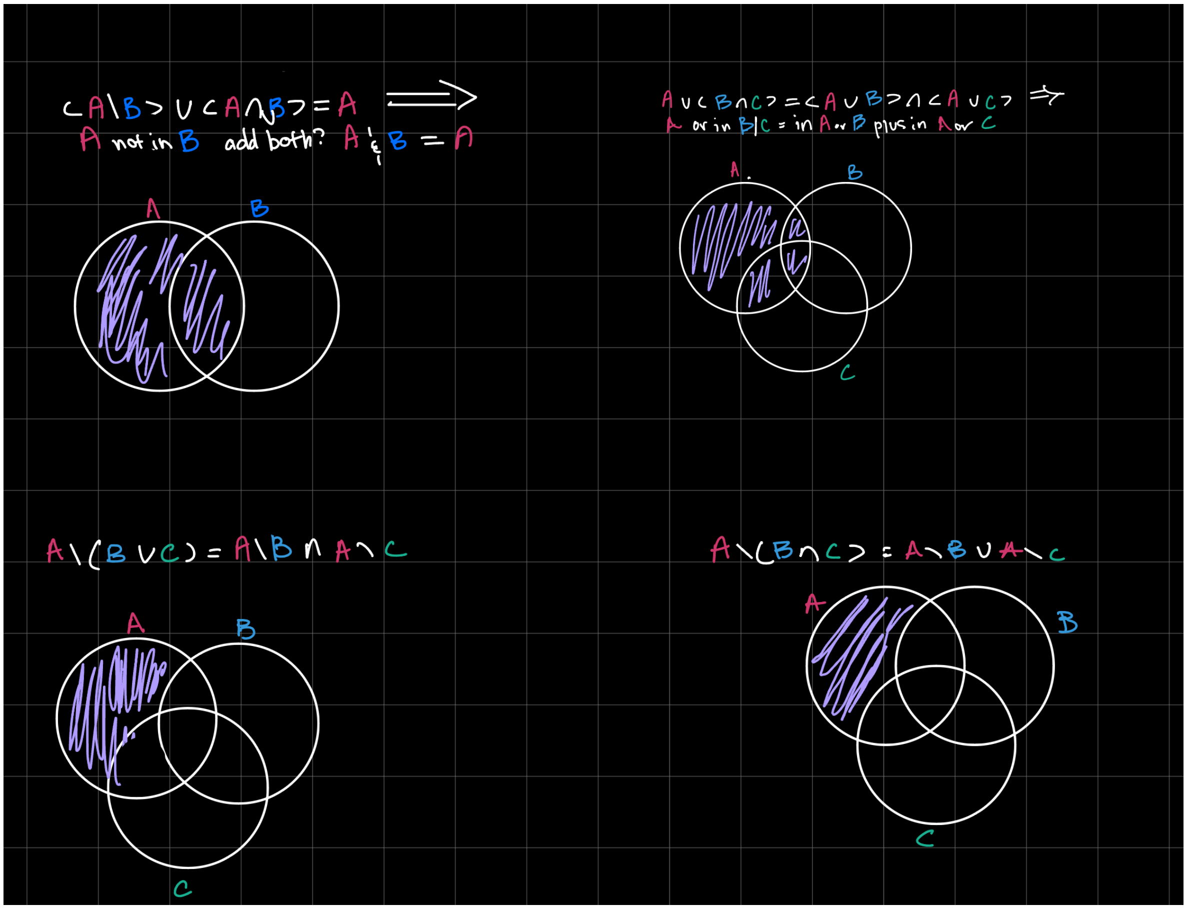
Do Question 6, and three more questions of your choice.

1. Set review:

- Operations on sets (hints: sketch venn-type diagrams, and showing $A = B$ is the same as $A \subseteq B$ and $B \subseteq A$):
- $(A \backslash B) \cup (A \cap B) = A$
- $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
- $A \backslash (B \cup C) = A \backslash B \cap A \backslash C$ and $A \backslash (B \cap C) = A \backslash B \cup A \backslash C$
- Plot the following sets:
  - $A = \{x \in \mathbb{R} : x^2 - 1 \geq 0\}$
  - $B = \{(x, y) \in \mathbb{R}^2 : 3x - 2y \geq 0\}$
  - $C = \{(x, y) \in \mathbb{R}^2 : xy \geq 3\}$
  - $D = \{(x_1, x_2) \in \mathbb{R}^2 : x_1 \geq 0, x_2 \geq 0, x_1 + x_2 \leq 1\}$
- The power set of A is the set of all subsets of A, denoted $\mathcal{P}(A)$. What is the power set of $\{1, 2, 3\}$? (Hint: The empty set is a subset of every set; the whole set is a subset of itself.)

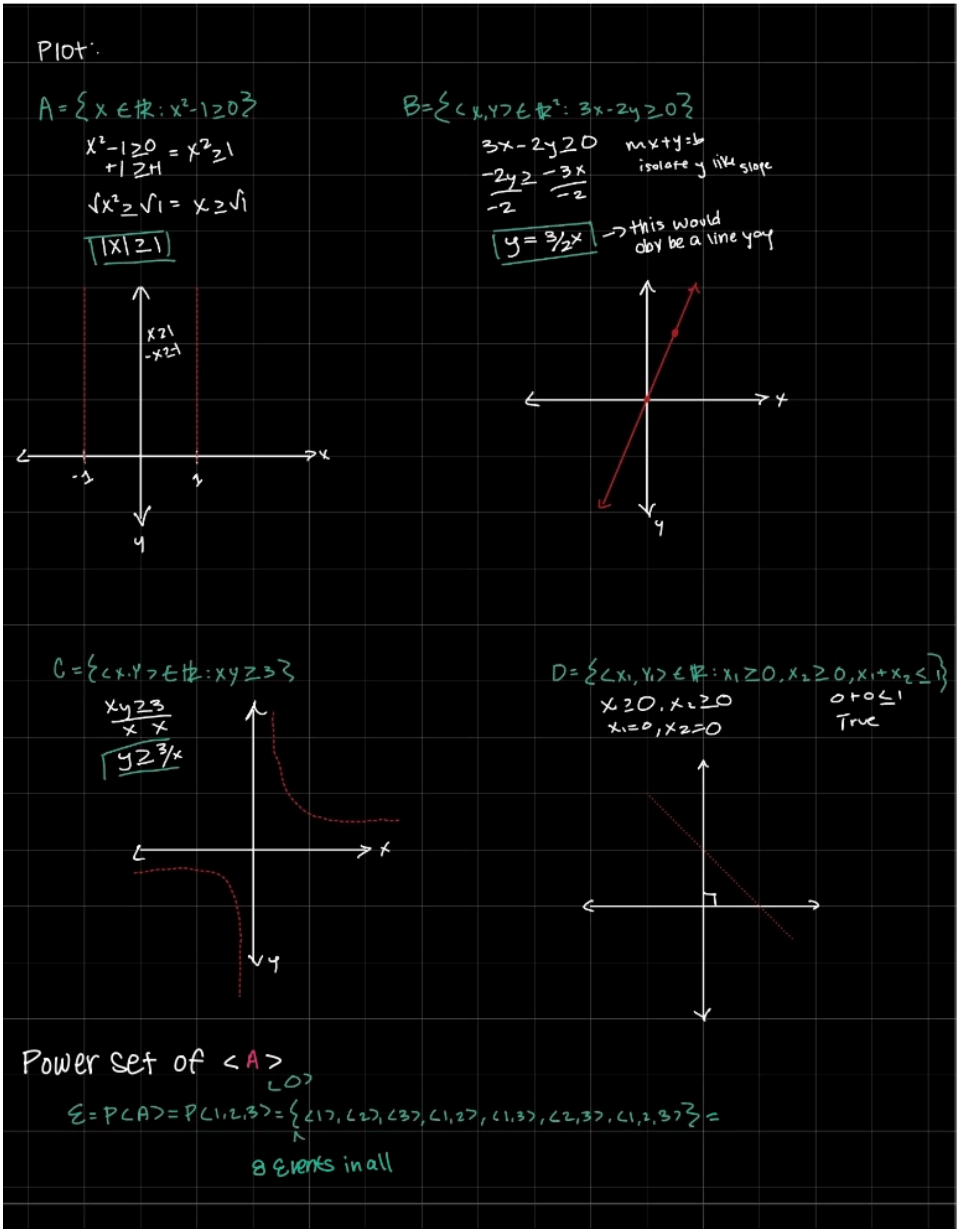
In [32]:
```python
from IPython.display import Image

image = '/Users/aeonlevy/Desktop/IMG_0213.jpg'
Image(filename=image)
```

Out[32]:



In [58]:
```python
image2 = '/Users/aeonlevy/Desktop/img2.jpg'
Image(filename=image2)
```

Out[58]:



^^^^THIS PART RIGHT AT THE BOTTOM IS THE POWER SET OF A

2. Probability space basics:

- What are the outcomes for rolling a fair **THREE**-sided die? What's the set of all events? What are the probabilities of all the events?
- What about flipping a fair coin twice? (Hint: There are 4 outcomes, and $2^4 = 16$ events.)

- What about rolling the **THREE**-sided die twice, and adding the results? Don't write down the set of all the possible events, but describe briefly what it looks like and how large it is. (Hint: There are 5 outcomes, and $2^5 = 32$ possible events.)

Obviously, a "three-sided die" doesn't exist, but this keeps you from spending a lot of time suffering in working out sets of events.

In [83]:
```
image3 = '/Users/aeonlevy/Desktop/img3.jpg'
Image(filename=image3)
```

Out[83]:

a) Three sided Die

outcome: $\{1,2,3\}$

Set of all Events: $\{\varnothing, \{1\}, \{2\}, \{3\}, \{1,2\}, \{2,3\}, \{1,3\}, \{1,2,3\}\} = 8$

Prob: $P(1) = 1/3$ $\quad P(1,2) = 1/3 + 1/3$ $\quad P(1,2,3) = 1/3 + 1/3 + 1/3$
$P(2) = 1/3$ $\quad P(2,3) = 1/3 + 1/3$
$P(3) = 1/3$ $\quad P(1,3) = 1/3 + 1/3$

b) Flip A coin!

outcome: {HT, TH, HH, TT}

set of all Events: 16

{{HT}, {TH}, {HH}, {TT}, {HT, TH}, {HT, TT}, {HT, HH}, {TH, TT}, {TH, HH}
{HH, TT}, {HH, HT, TH}, {HH, TH, TT}, {HH, HT, TT}, {HT, TH, TT}, {HH, HT, TH, TT}}

Prob: P(HH) 1/4 $\quad$ P(HH, HT, TH) = 3/4
P(TT) = 1/4 $\quad$ P(HH, TH, TT) = 3/4
P(HT) = 1/4 $\quad$ P(HH, HT, TT) = 3/4
P(TH) = 1/4 $\quad$ P(HT, TH, TT) = 3/4
P(HH, TT) = 2/4 $\quad$ P(all) = 1 = 4/4
P(HH, TH) = 2/4
P(HH, HT) = 2/4 } 1/2
P(TT, TH) = 2/4
P(TT, HT) = 2/4

c) Dye x 2 $\quad$ $\overset{2}{1+1}, \overset{3}{1+2}, \overset{4}{1+3}, \overset{4?}{2+2}, \overset{5}{2+3}, 3+3, 2+1, 3+1, 3+2$

outcomes: $\{2, 3, 4, 5, 6\}$ sum $\quad$ $3 \times 3 = 9$ $\quad$ $2^9 = 512$ solo

Set of event: Very big: 32 $\quad$ 5 sums = $2^5 = 32$ with sum

Prob: $P(2) = 1/9$ $\quad P(4) = 3/9$
$P(3) = 2/9$ $\quad P(5) = 2/9$
$P(6) = 1/9$

3. Random Variable Basics

- Imagine rolling a fair single six-sided die. There are 6 outcomes, all equally likely. Derive the sample space and the space of events. What are the probabilities of the outcomes and events?
- Consider a random variable that assigns the square root of the number of

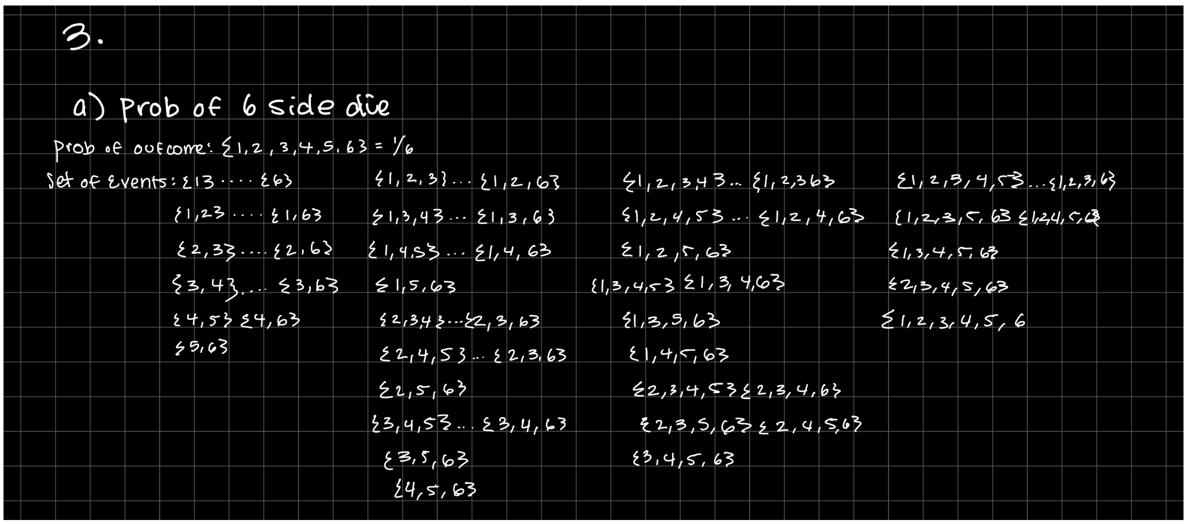
pips on the die to each outcome. Write code to simulate rolling a single six-sided die and computing the value of the random variable. Simulate 5000 rolls and plot the mass function and ECDF of the random variable.

- Imagine rolling two fair six-sided die. Consider a random variable that adds up the pips on the dice. There are 11 outcomes (2 , 3, ..., 12), but not all are equally likely. Derive the sample space and **describe** the space of events. What are the probabilities of the outcomes?
- Write code to simulate the random variable (rolling two six-sided die and adding the results together). Simulate 10000 rolls and plot the mass function and ECDF.

In [179…

```
image4 = '/Users/aeonlevy/Desktop/IMG_0277.jpg'
Image(filename=image4)
```

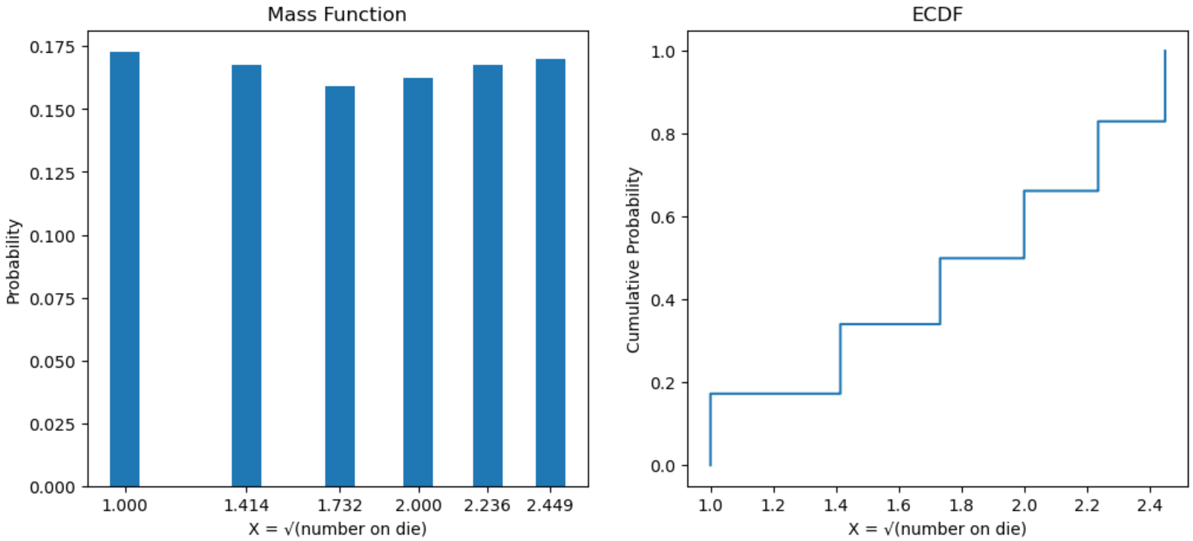
Out[179…



In [180…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

n = 5000
rolls = np.random.randint(1, 7, size=n)

X = np.sqrt(rolls)

pmf = pd.Series(X).value_counts(normalize=True).sort_index()

X_sorted = np.sort(X)
ecdf = np.arange(1, n + 1) / n

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.bar(pmf.index, pmf.values, width=0.1)
plt.title("Mass Function")
plt.xlabel("X = √(number on die)")
```

```
plt.ylabel("Probability")
plt.xticks(pmf.index)

plt.subplot(1, 2, 2)
plt.step(X_sorted, ecdf, where='post')
plt.title("ECDF")
plt.xlabel("X = √(number on die)")
plt.ylabel("Cumulative Probability")
plt.show()
```

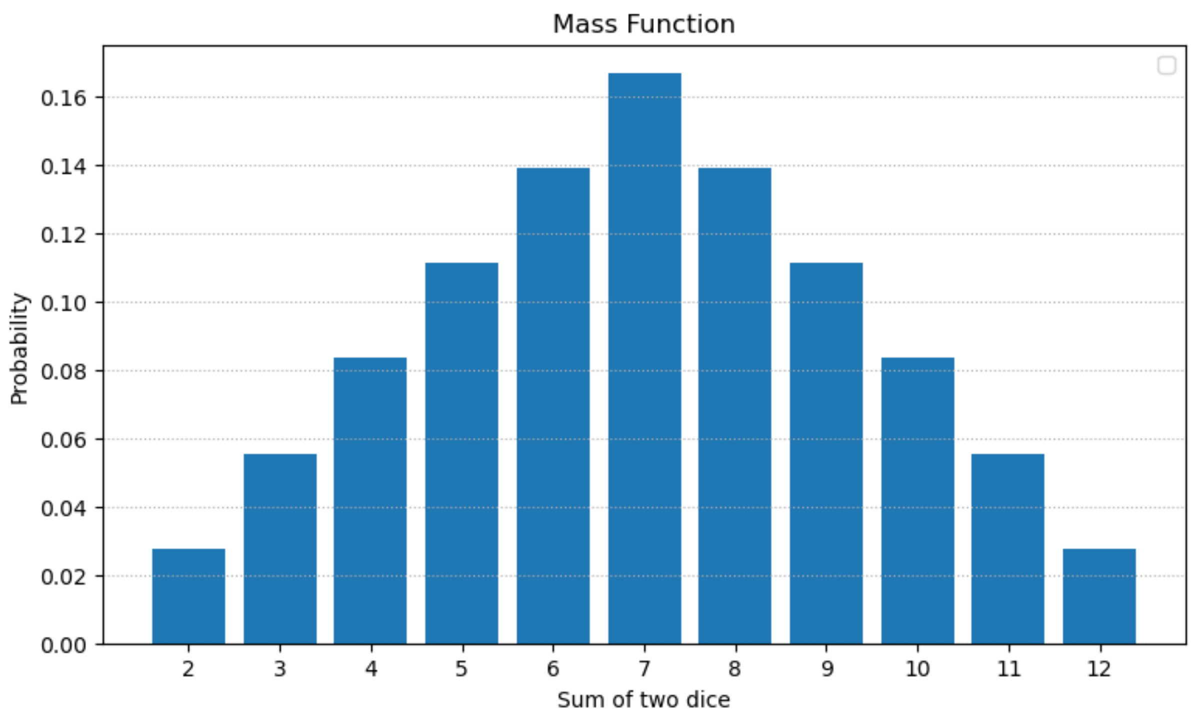


In [181…

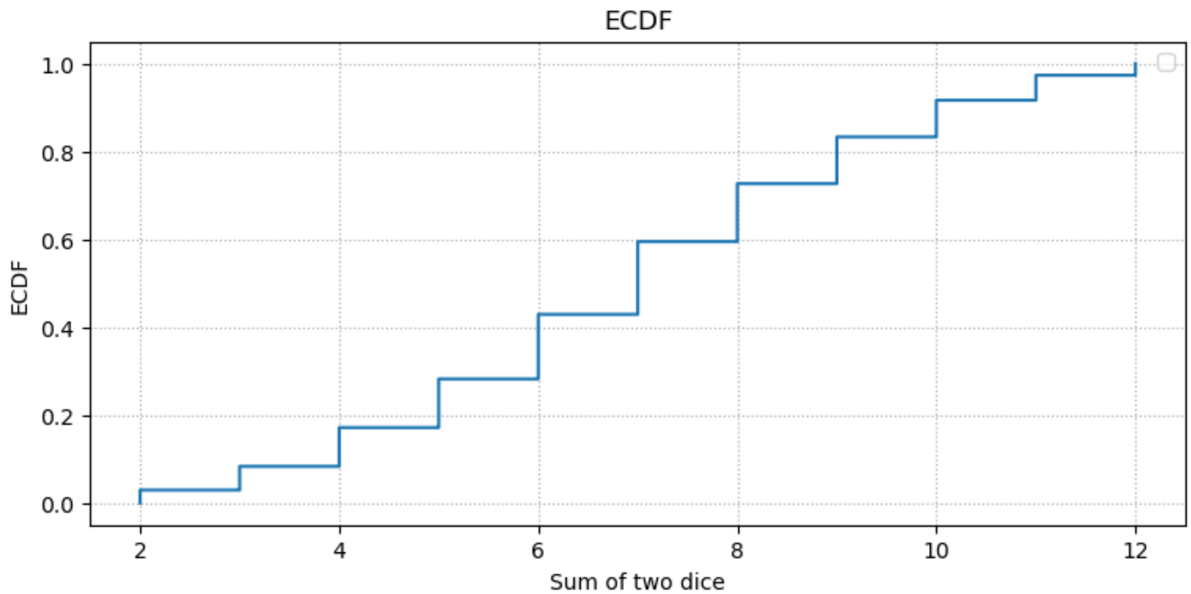
```
rolls = 10000
rng = np.random.default_rng(42)

dice1 = rng.integers(1, 7, size=rolls)
dice2 = rng.integers(1, 7, size=rolls)
sums = dice1 + dice2

values = np.arange(2, 13)
counts = np.array([np.sum(sums == v) for v in values])
probs = counts / rolls

counts = np.array([1,2,3,4,5,6,5,4,3,2,1])
probs = counts / 36


plt.figure(figsize=(9,5))
plt.bar(values, probs)
plt.xticks(values)
plt.xlabel("Sum of two dice")
plt.ylabel("Probability")
plt.title("Mass Function")
plt.legend()
plt.grid(axis='y', linestyle=':')
plt.show()

sorted_sums = np.sort(sums)
```

```
ecdf = np.arange(1, rolls+1)/rolls

plt.figure(figsize=(9,4))
plt.step(sorted_sums, ecdf)
plt.xlabel("Sum of two dice")
plt.ylabel("ECDF")
plt.title("ECDF")
plt.grid(linestyle=':')
plt.legend()
plt.show()
```

```
/var/folders/2q/0pm84ptn56b3zt11wj9sryf80000gn/T/ipykernel_57426/212874
1352.py:22: UserWarning: No artists with labels found to put in legend.
Note that artists whose label start with an underscore are ignored when
legend() is called with no argument.
  plt.legend()
```




```
/var/folders/2q/0pm84ptn56b3zt11wj9sryf80000gn/T/ipykernel_57426/212874
1352.py:35: UserWarning: No artists with labels found to put in legend.
Note that artists whose label start with an underscore are ignored when
legend() is called with no argument.
  plt.legend()
```

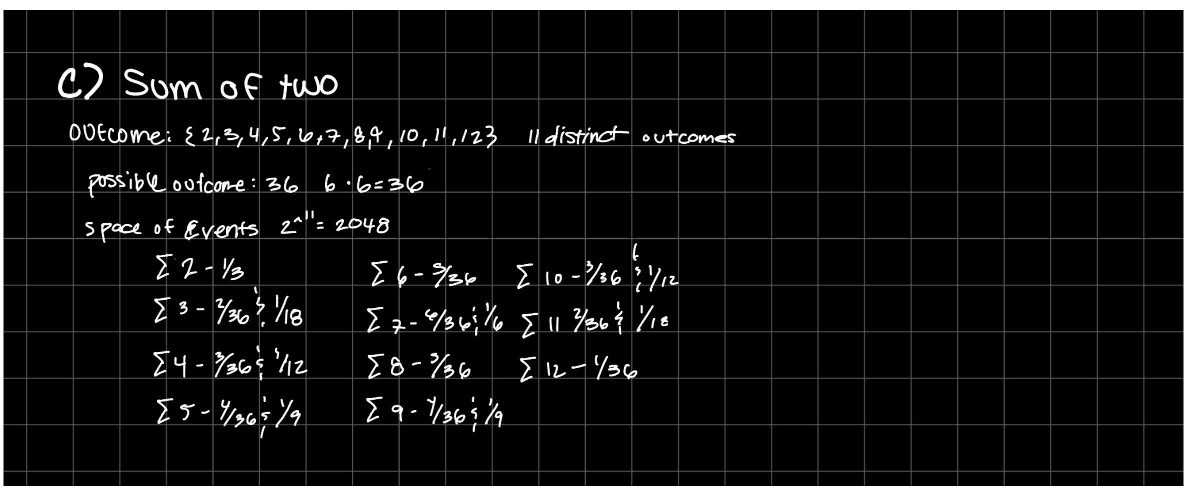


- Consider a random variable When rolling two dice, an outcome is an ordered pair (i,j), each value is from 1–6. Therefore the sample space is S=(i,j)(1,2,3,4,5,6). There are 36 possible outcomes, each with probability 1/36. For any event

In [182…

```
image5 = '/Users/aeonlevy/Desktop/IMG_0278.jpg'
Image(filename=image5)
```

Out[182…



4. Roulette

This question will be easiest if you read the whole thing, and come up with a clear plan for how you'll write the code.

Roulette is a betting game. There are 37 possible outcomes: A green 0, and the numbers 1 to 36 in red and black. Here is a picture of the (American, not Euro, it has an extra green 00) betting board:

Roulette

To bet, you must pay a dollar, but then you get payouts that depend on how many slots are in your bet

- Basic bets:
  - Red or Black slots
  - Odd or Even slots
  - A single slots, like 20
- More complex bets:
  - Split: Two adjacent slots (e.g. {1,2})
  - Square: Four adjacent slots (e.g. {1,2,4,5})
  - Street: Three slots in a row (e.g. {1,2,3})
  - Line: Six slots (e.g. {1,2,3,4,5,6}) In general, you can only bet on 1, 2, 3, 4, 6 slots, 12 slots, or 18 slots. If your bet occurs when the wheel is spun, you gain 36/K-1 where K is the number of slots you bet on; if not, you lose a dollar and get -1.
- Write code to model spinning the roulette wheel, including the colors and numbers (you could make two lists of number and color and draw a random number between 0 and 37... or use a dataframe with color and number variables and sample it... or use a dict with key to number/color pairs...)

- Describe the probability space associated with the roulette wheel: Outcomes, events, probabilities (If there are 37 outcomes, there are $2^{37} = 137,438,953,472$ events, by the way)

- You wrote code to generate a spin of the roulette wheel. Now write a function that takes a basic or complex bet as an argument, and returns the result for the player (win or lose, and the payout 36/K-1 or -1)

- Simulate betting on red, betting on odd, betting on 7, a split, and a line 1000 times each.

- Compute the average values for the bets you just simulated. What are the expected average payoffs?

5. CDF and PDF Basics

- Verify the following functions are distribution functions and compute their density functions. Plot the distribution and density.

1. $F(x) = \begin{cases} 0, & x \leq 0 \\ \sqrt{x}, & 0 \leq x \leq 1 \\ 1, & x \geq 1 \end{cases}$
2. $F(x) = \dfrac{1}{1+e^{-x}}$
3. For $a < b < c$, $F(x) = \begin{cases} 0, & x \leq 0 \\ \frac{(x-a)^2}{(b-a)(b-c)}, & a \leq x \leq c \\ 1 - \frac{(b-x)^2}{(b-a)(b-c)}, & c < x < b \\ 1, & x \geq b \end{cases}$
4. $F(x) = \begin{cases} 0, & x < 0 \\ 1 - e^{-x}, & x > 0 \end{cases}$
5. $F(x) = \begin{cases} 0, & x \leq 0 \\ x, & 0 < x < 1 \\ 1, & x \geq 1 \end{cases}$

6. Some Common Distributions

For the following distributions:

- Determine the support
- Compute the density from the distribution for the logistic and exponential distributions (take a derivative)
- Plot the density and distribution for a variety of parameter values
- Take a sample of 1000 draws $(x_1, x_2, \ldots, x_{1000})$ from the distribution, plot a KDE and ECDF, visually compare with the theoretical pdf/cdf
- Find an example of this general type of PDF/CDF from the Metabric cancer data

You can use https://docs.scipy.org/doc/scipy/reference/stats.html to generate values for the pdf/cdf and generate samples of random variates.

- Logistic distribution (similar to normal):

$$F(x; \sigma) = \frac{1}{1 + e^{-x/\sigma}}$$

  with $\sigma > 0$.
- Exponential distribution (similar to log-normal):

$$F_X(x) = \begin{cases} 0, & x < 0 \\ 1 - e^{-\lambda x}, & x \geq 0, \end{cases}$$

  with $\lambda > 0$.

- Negative Binomial (similar to Poisson): The probability mass function for positive integers is:

$$f(k; r, p) = \frac{(k + r - 1)!}{k!(r - 1)!}(1 - p)^k p^r, \qquad \text{for } k = 0, \ldots n$$

  You can interpret this as follows: Flip a coin that comes up heads with probability p until you get r heads, and then stop. What is the probability of stopping at each $k = 0, 1, 2, \ldots$?
- Categorical (similar to Bernoulli): The probability mass function over $k = 1, 2, \ldots, K$ categories is

$$f(k; p_1, \ldots, p_K) = p_1^{k=1} p_2^{k=2} \ldots p_K^{k=K}$$

  where $0 \leq p_i \leq 1$ and $\sum_{k=1}^{K} p_k = 1$.

In [183…

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy as sp
```

LOG DISTRUBUTION

In [184…

```
from scipy.stats import logistic
```

In [185…

```
##SUPPORT
lb, ub = logistic.support()
lb, ub
```

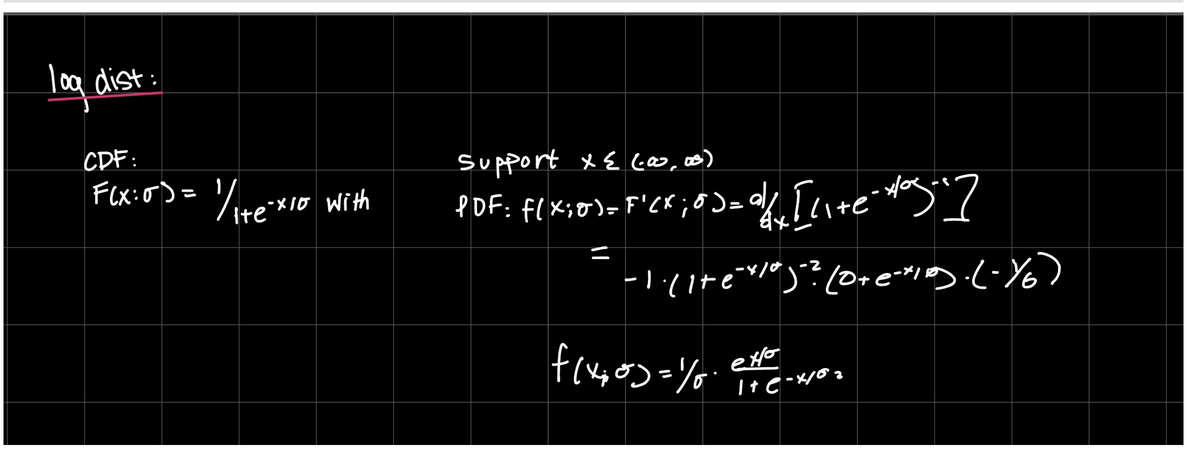
Out[185…

```
(np.float64(-inf), np.float64(inf))
```

In [186…

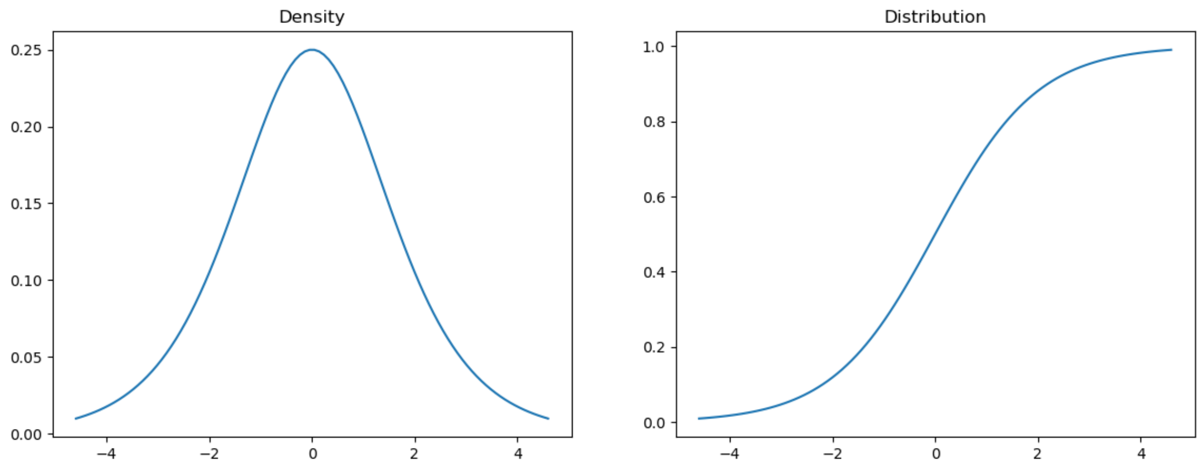
```
image6 = '/Users/aeonlevy/Desktop/IMG_0214.jpg'
Image(filename=image6)
```

Out[186…



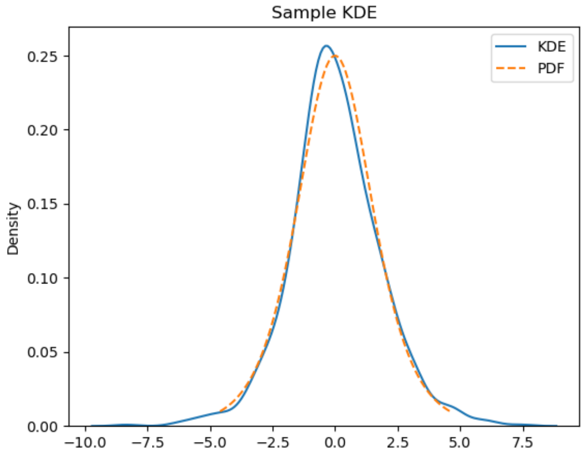
In [187…

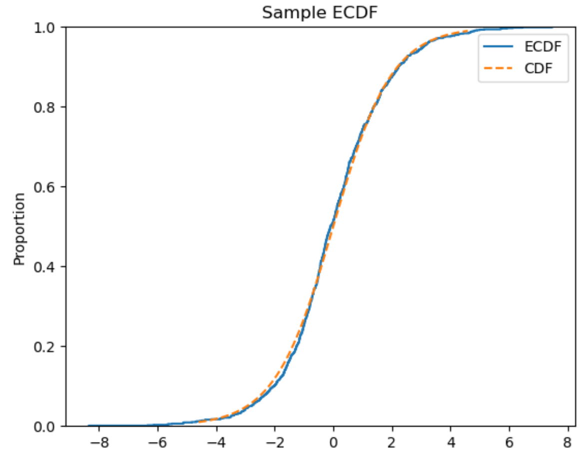
```
###PLOTTING

grid = np.linspace(logistic.ppf(0.01),
                   logistic.ppf(0.99), 100)

# Plot the density and distribution
pdf = logistic.pdf(grid)
cdf = logistic.cdf(grid)

fig, axes = plt.subplots(1, 2, figsize = (14, 5))
sns.lineplot(x = grid, y = pdf, ax = axes[0])
axes[0].set_title('Density')
sns.lineplot(x = grid, y = cdf, ax = axes[1])
axes[1].set_title('Distribution')
plt.show()
```



In [188…

```
##KDE AND ECDF
# Sample of 1000 draws
sample = logistic.rvs(size = 1000)

fig, axes = plt.subplots(1, 2, figsize = (14, 5))
sns.kdeplot(x = sample, ax = axes[0], label = 'KDE')
sns.lineplot(x = grid, y = pdf, ax = axes[0], label = 'PDF', linestyle
axes[0].set_title('Sample KDE')
sns.ecdfplot(x = sample, ax = axes[1], label = 'ECDF')
sns.lineplot(x = grid, y = cdf, ax = axes[1], label = 'CDF', linestyle
axes[1].set_title('Sample ECDF')
plt.show()
```





EXAMPLE: One example of this from the Metabric data could be... Maybe age or age at diagonis

EXPONENTIAL DISTRIBUTION

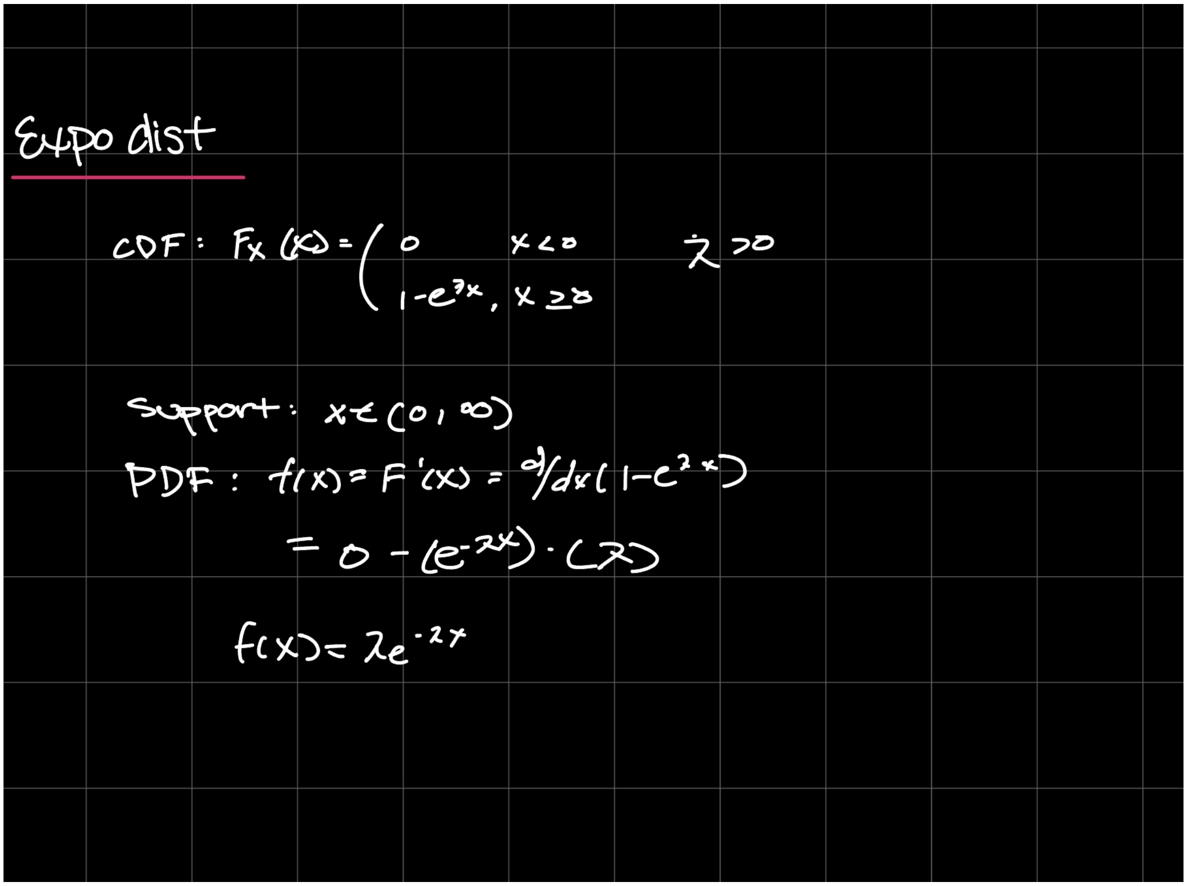
In [189…
```
from scipy.stats import expon
```

In [190…
```
#SUPPORT
lb, ub = expon.support()
lb, ub
```

Out[190…
```
(np.float64(0.0), np.float64(inf))
```

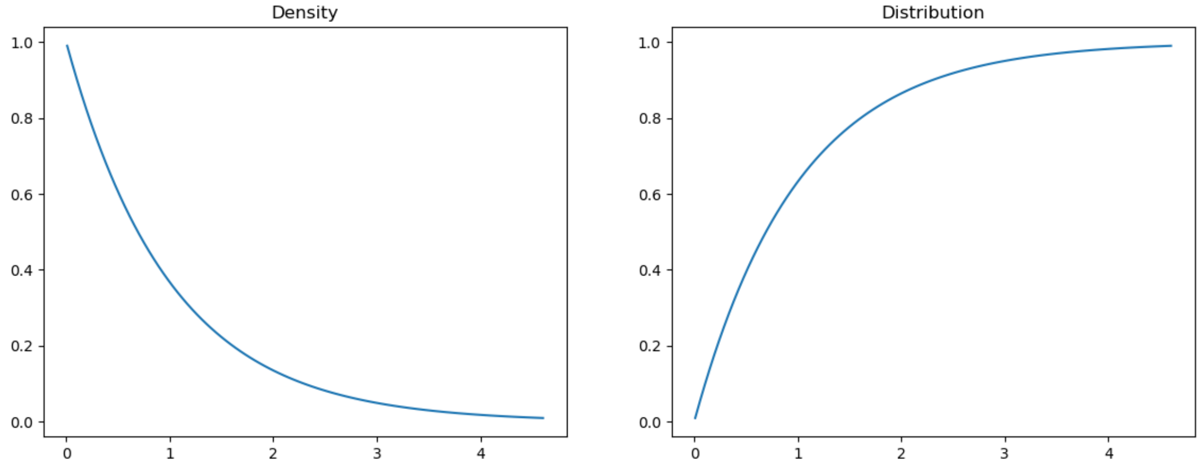
In [191…
```
image7 = '/Users/aeonlevy/Desktop/IMG_0215.jpg'
Image(filename=image7)
```

Out[191…

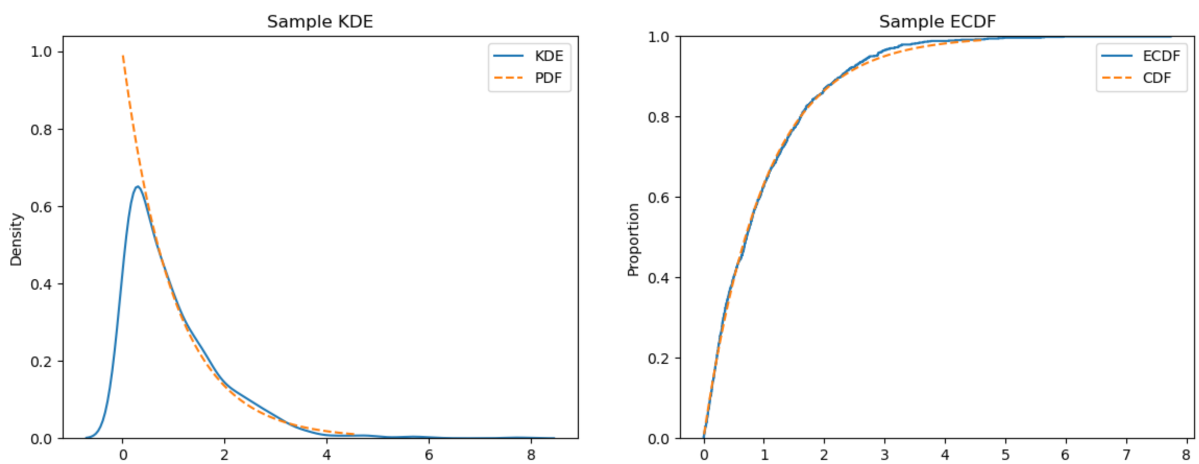


In [192…

```
##PLOTTING
grid = np.linspace(expon.ppf(0.01),
                   expon.ppf(0.99), 100)

# Plot the density and distribution
pdf = expon.pdf(grid)
cdf = expon.cdf(grid)

fig, axes = plt.subplots(1, 2, figsize = (14, 5))
sns.lineplot(x = grid, y = pdf, ax = axes[0])
axes[0].set_title('Density')
sns.lineplot(x = grid, y = cdf, ax = axes[1])
axes[1].set_title('Distribution')
plt.show()
```



In [193…
```
##KDE AND ECDF
# Sample of 1000 draws
sample = expon.rvs(size = 1000)

fig, axes = plt.subplots(1, 2, figsize = (14, 5))
sns.kdeplot(x = sample, ax = axes[0], label = 'KDE')
sns.lineplot(x = grid, y = pdf, ax = axes[0], label = 'PDF', linestyle
axes[0].set_title('Sample KDE')
sns.ecdfplot(x = sample, ax = axes[1], label = 'ECDF')
sns.lineplot(x = grid, y = cdf, ax = axes[1], label = 'CDF', linestyle
axes[1].set_title('Sample ECDF')
plt.show()
```



EXAMPLE: survival or tumor size

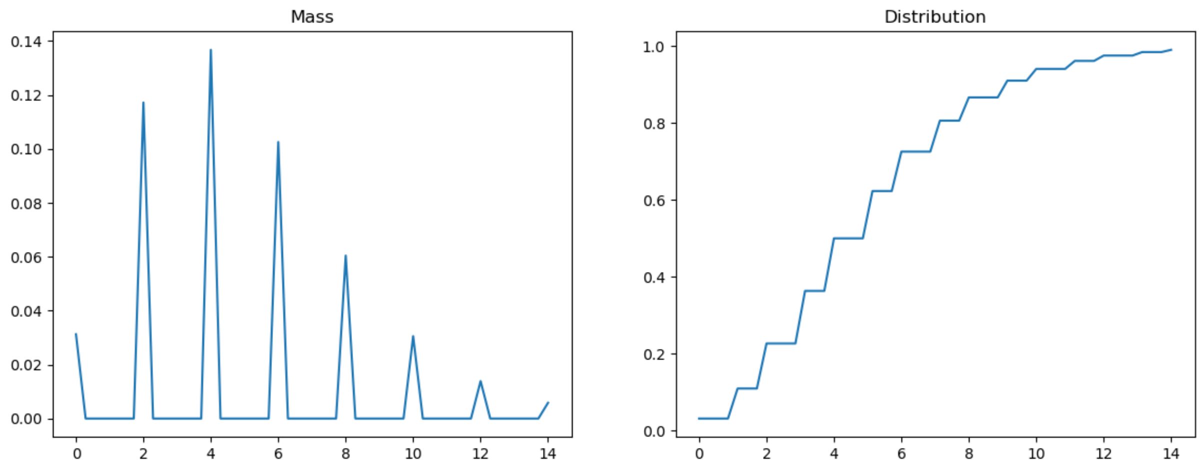
NEGATIVE BINOMIAL

In [194…
```
from scipy.stats import nbinom
```

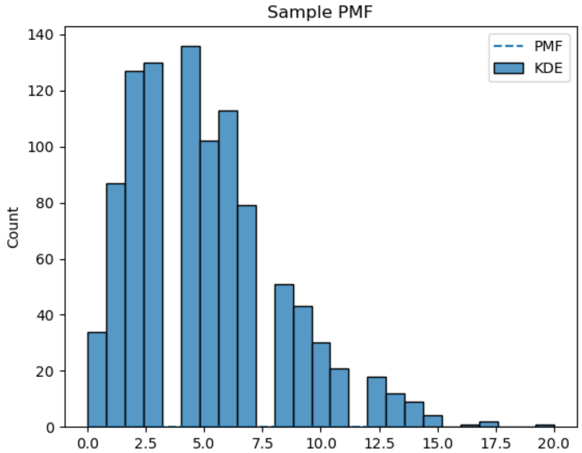
In [195…
```
#SUPPORT
n, p = 5, 0.5
lb, ub = nbinom.support(n, p)
lb, ub
```

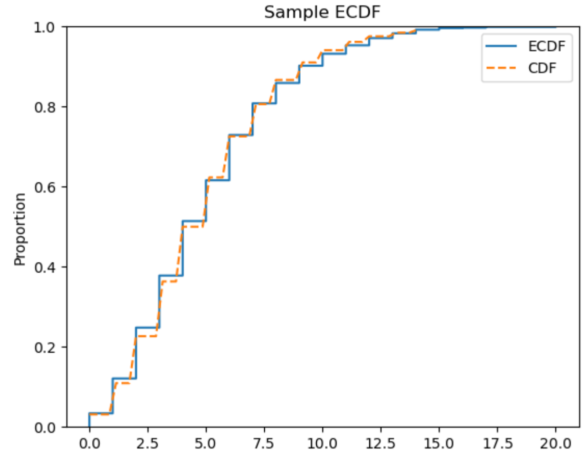
Out[195… (np.int64(0), np.float64(inf))

In [196…
```
#PLOTTING
grid = np.linspace(nbinom.ppf(0.01, n, p),
                   nbinom.ppf(0.99, n, p))

# Plot the probability mass function and distribution
pmf = nbinom.pmf(grid, n, p)
cdf = nbinom.cdf(grid, n, p)

fig, axes = plt.subplots(1, 2, figsize = (14, 5))
sns.lineplot(x = grid, y = pmf, ax = axes[0]) # or sns.barplot(x = gri
axes[0].set_title('Mass')
sns.lineplot(x = grid, y = cdf, ax = axes[1])
axes[1].set_title('Distribution')
plt.show()
```



In [197…
```
##KDE AND ECDF
# Sample of 1000 draws
sample = nbinom.rvs(n, p, size = 1000)

fig, axes = plt.subplots(1, 2, figsize = (14, 5))
sns.histplot(x = sample, ax = axes[0], label = 'KDE')
sns.lineplot(x = grid, y = pmf, ax = axes[0], label = 'PMF', linestyle
axes[0].set_title('Sample PMF')
sns.ecdfplot(x = sample, ax = axes[1], label = 'ECDF')
sns.lineplot(x = grid, y = cdf, ax = axes[1], label = 'CDF', linestyle
axes[1].set_title('Sample ECDF')
plt.show()
```





EXAMPLE:Something with counts

CATEGORICAL

In [198…

```
from scipy.stats import bernoulli
```

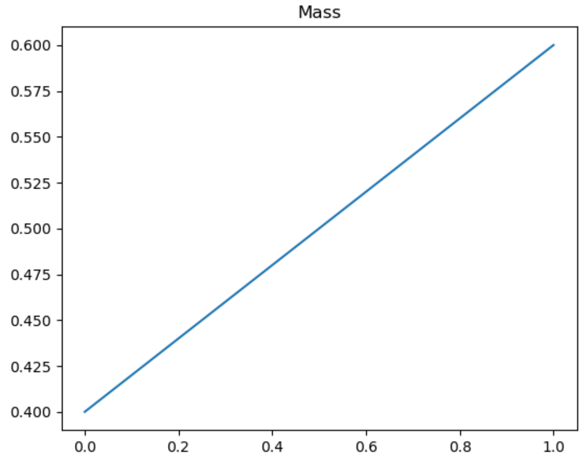
In [199…

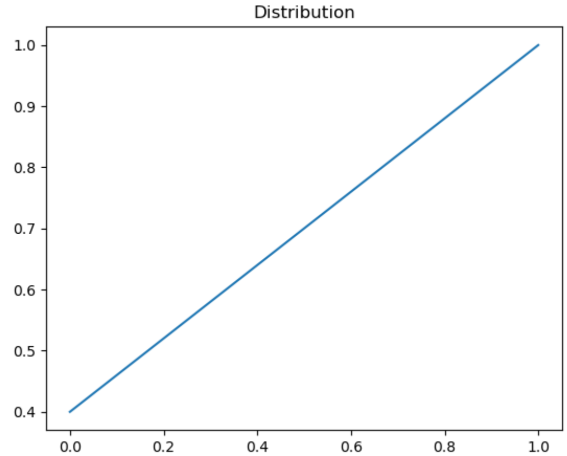
```
###SUPPORT
p = 0.6
dist = bernoulli(p)

# support = possible values that can occur
support = dist.support()

print("Support:", support)
```

```
Support: (np.int64(0), np.int64(1))
```

In [200…

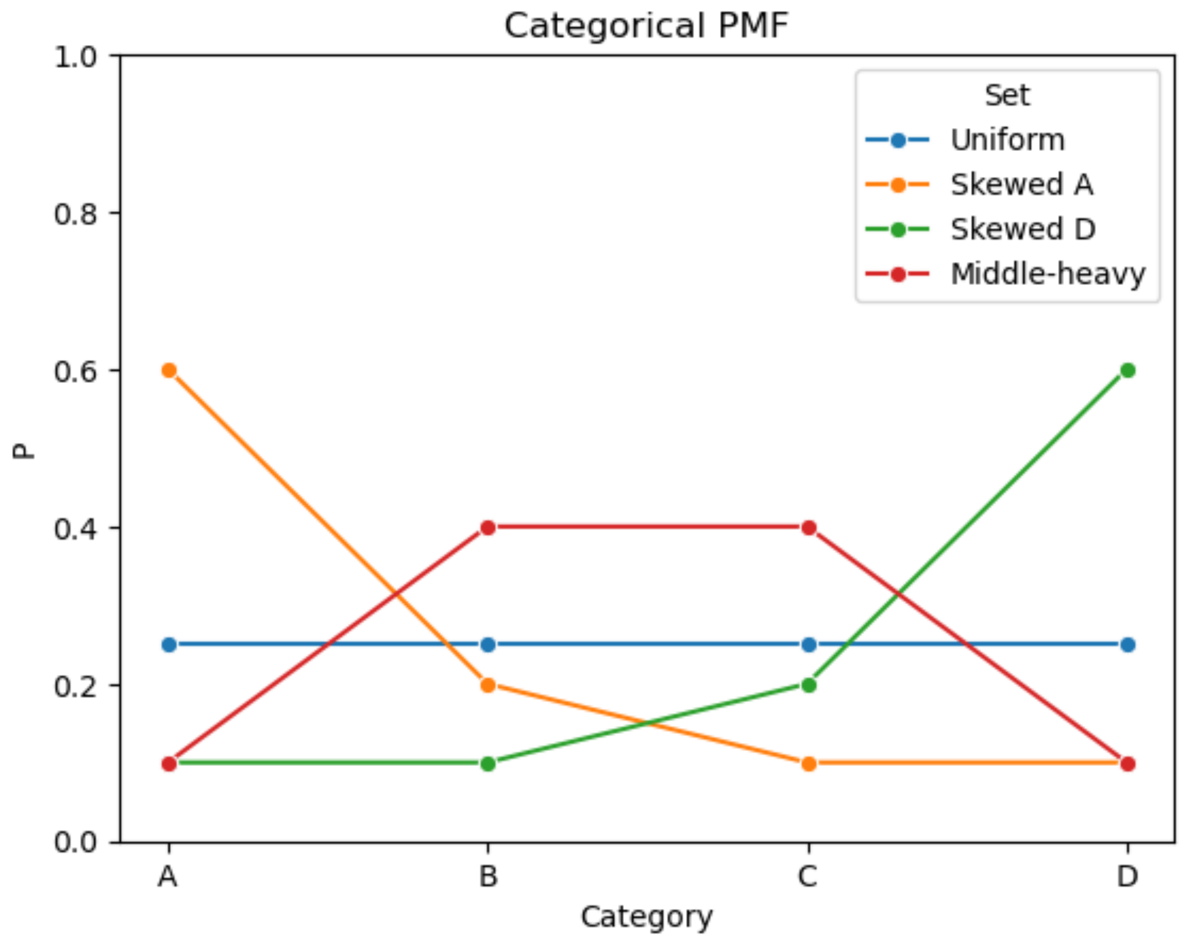
```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as

##PLOTTING
# Plot the probability mass function and distribution
pmf = dist.pmf(support)
cdf = dist.cdf(support)

fig, axes = plt.subplots(1, 2, figsize = (14, 5))
sns.lineplot(x = support, y = pmf, ax = axes[0])
axes[0].set_title('Mass')
sns.lineplot(x = support, y = cdf, ax = axes[1])
axes[1].set_title('Distribution')
plt.show()
```

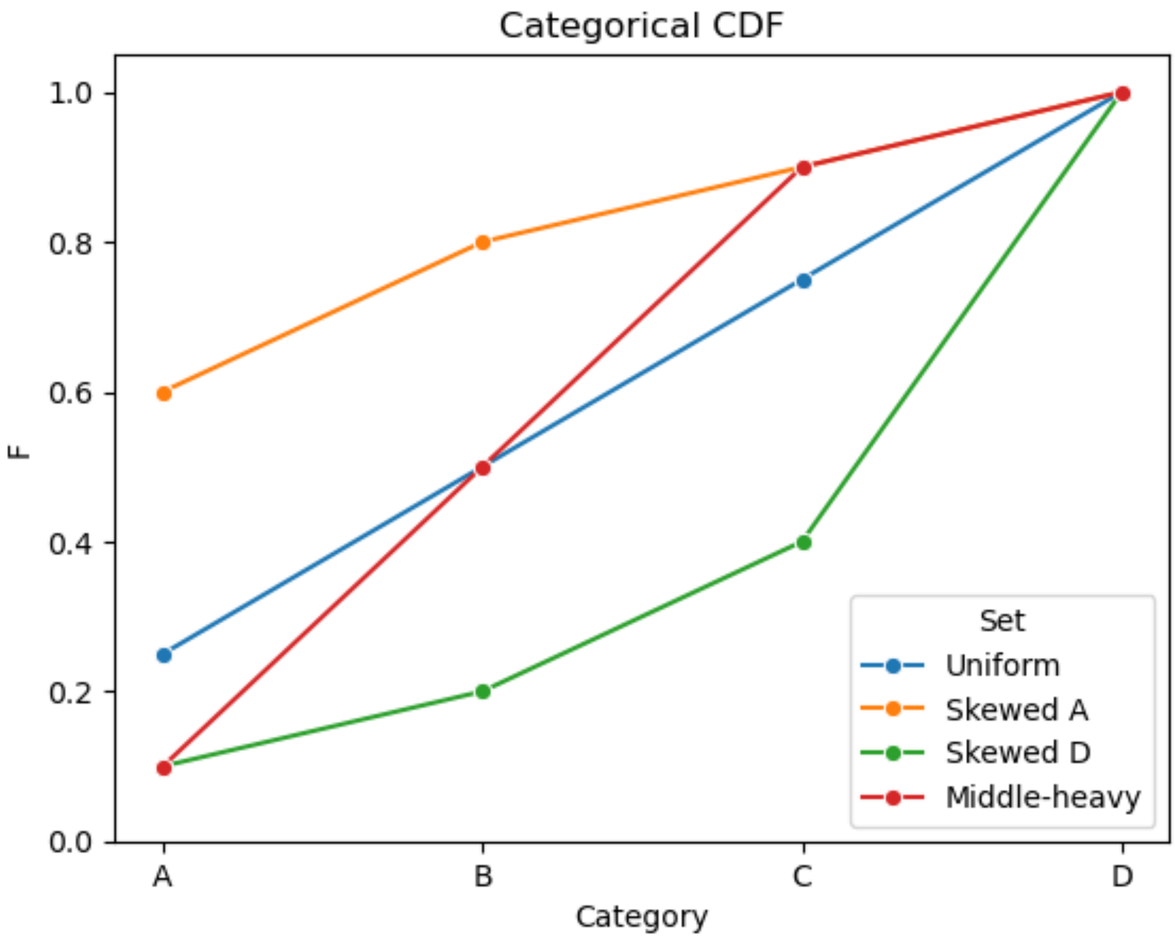




In [201…

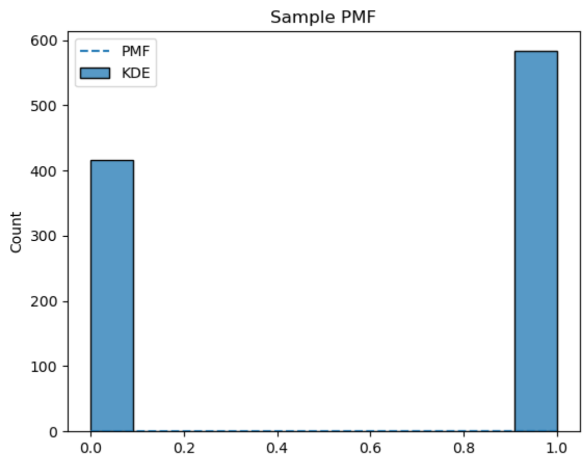
```
cats = ["A","B","C","D"]
param_sets = {
    "Uniform":[.25,.25,.25,.25],
    "Skewed A":[.6,.2,.1,.1],
    "Skewed D":[.1,.1,.2,.6],
    "Middle-heavy":[.1,.4,.4,.1]
}

# Build tidy DataFrame for PMF and CDF
pmf_df = pd.DataFrame([(n,c,p) for n,v in param_sets.items() for c,p i
                      columns=["Set","Category","P"])
cdf_df = pmf_df.copy()
cdf_df["F"] = cdf_df.groupby("Set")["P"].cumsum()

# PMF
sns.lineplot(data=pmf_df, x="Category", y="P", hue="Set", marker="o")
plt.title("Categorical PMF"); plt.ylim(0,1); plt.show()

# CDF
sns.lineplot(data=cdf_df, x="Category", y="F", hue="Set", marker="o")
plt.title("Categorical CDF"); plt.ylim(0,1.05); plt.show()
```

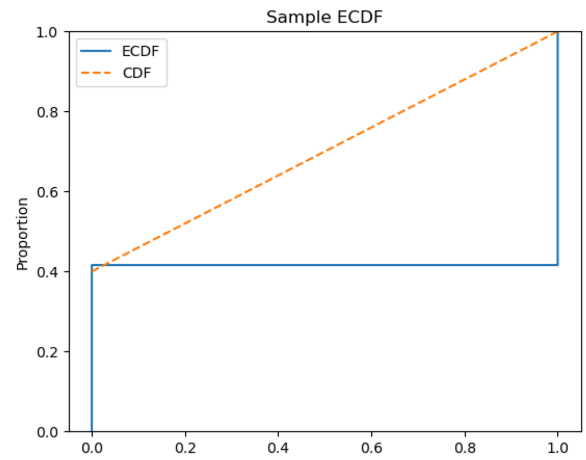

Categorical PMF
1.0
0.8
0.6
0.4
0.2
0.0
P
A
B
C
D
Category
Set
Uniform
Skewed A
Skewed D
Middle-heavy



In [202…

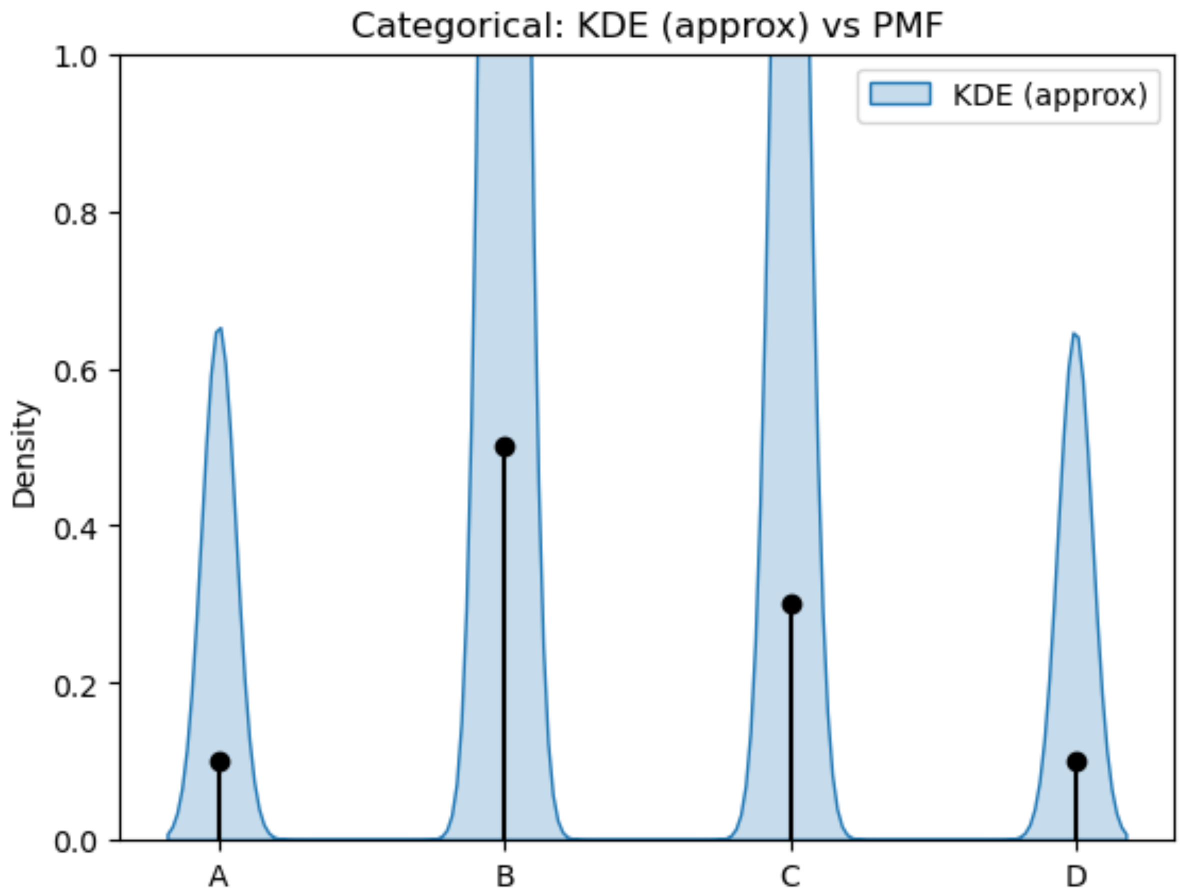
```
##KDE AND ECDF
# Sample of 1000 draws
sample = bernoulli.rvs(p, size = 1000)

fig, axes = plt.subplots(1, 2, figsize = (14, 5))
sns.histplot(x = sample, ax = axes[0], label = 'KDE')
sns.lineplot(x = support, y = pmf, ax = axes[0], label = 'PMF', linest
axes[0].set_title('Sample PMF')
sns.ecdfplot(x = sample, ax = axes[1], label = 'ECDF')
sns.lineplot(x = support, y = cdf, ax = axes[1], label = 'CDF', linest
axes[1].set_title('Sample ECDF')
plt.show()
```

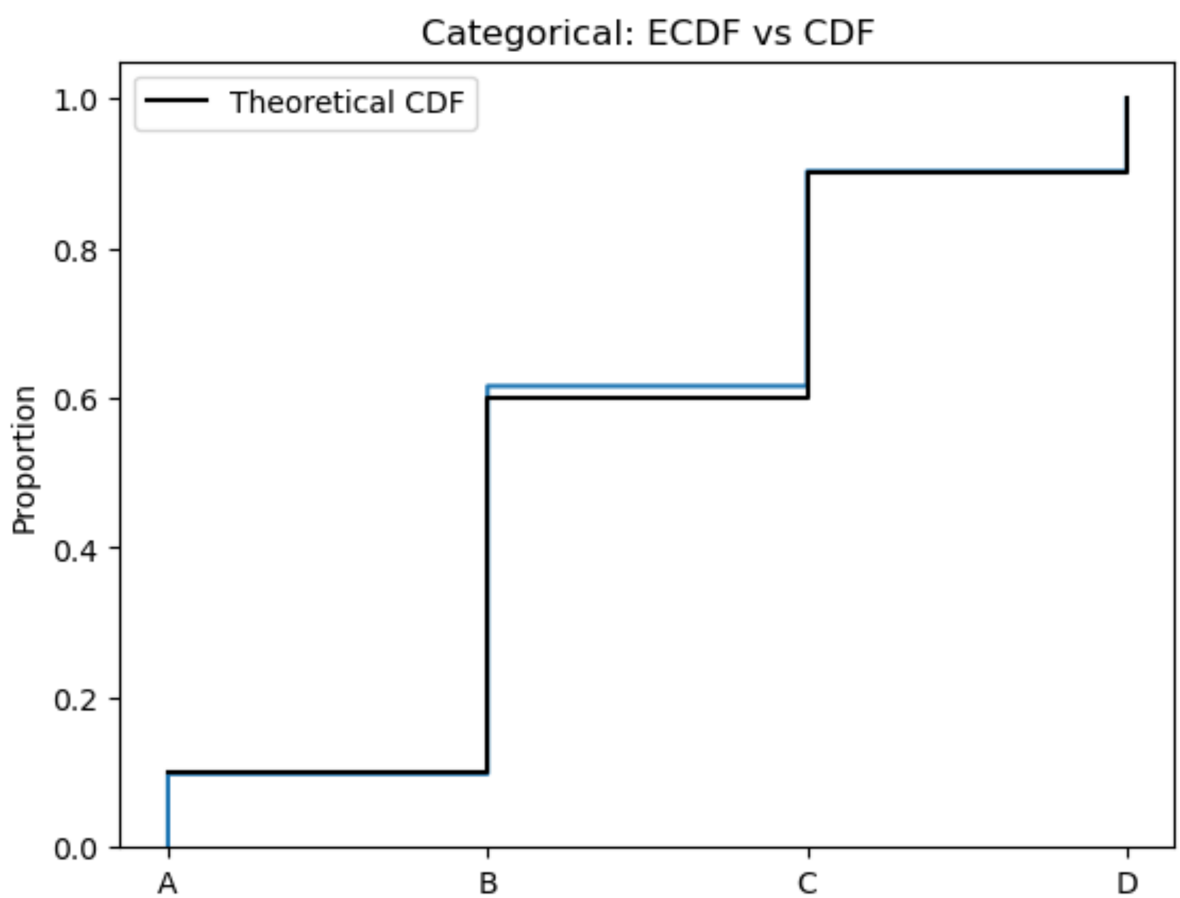




In [203…

```
import numpy as np, seaborn as sns, matplotlib.pyplot as plt

cats = ["A","B","C","D"]
p = [0.1,0.5,0.3,0.1]
idx = np.arange(len(cats))

# theoretical pmf/cdf
cdf = np.cumsum(p)

# sample 1000 draws
samp = np.random.choice(idx, size=1000, p=p)

# KDE vs theoretical PMF
sns.kdeplot(samp, bw_adjust=0.3, fill=True, label="KDE (approx)")
plt.stem(idx, p, linefmt="k-", markerfmt="ko", basefmt=" ")
plt.xticks(idx, cats); plt.ylim(0,1)
plt.title("Categorical: KDE (approx) vs PMF")
plt.legend(); plt.show()

# ECDF vs theoretical CDF
sns.ecdfplot(samp, stat="proportion")
plt.step(idx, cdf, where="post", color="k", label="Theoretical CDF")
plt.xticks(idx, cats); plt.ylim(0,1.05)
plt.title("Categorical: ECDF vs CDF")
plt.legend(); plt.show()
```





EXAMPLE:Cases Like Yes or No